

Piloto de una red de monitoreo y alerta temprana de huaicos basada en nodos IoT con conectividad híbrida terrestre-satelital en la cuenca del río Rímac

Arteaga Ayca Adrian Eliseo , Shiguay Vargas Illary Ariadna

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Lima, Perú

adrian.arteaga@unmsm.edu.pe

illary.shiguay@correo.Com

Resumen

Las limitaciones de las infraestructuras de telecomunicaciones en la cuenca del río Rímac indican una vulnerabilidad de estado crítico si suceden desastres como los huaicos, porque existe el riesgo de cortes físicos que llegan a aislar los sistemas de monitoreo del Centro de Operaciones de Emergencia Nacional (COEN). En este informe presentamos el diseño, simulación y validación de una arquitectura de red lógica híbrida tolerante a fallos para una transmisión ininterrumpida de telemetría de alerta temprana utilizando GNS3. La solución propuesta une la segmentación de tráfico mediante redes locales virtuales (VLAN), redundancia de enlace en Capa 2 con el protocolo Spanning Tree (RSTP), enrutamiento dinámico OSPF y el transporte de datos IoT mediante el protocolo MQTT sobre un broker Eclipse Mosquitto simulado en contenedores Alpine Linux. Las validaciones de tráfico realizadas con Wireshark indican que la segmentación de red aísla eficazmente los datos críticos de los sensores frente al tráfico multimedia IP. También, las pruebas de conmutación lógica confirman tiempos de convergencia ante fallas inferiores a los 2 segundos, esto garantiza la alta disponibilidad del sistema de alerta temprana cuando ocurran contingencias severas. *Palabras clave*— IoT, MQTT, RSTP, VLAN, GNS3, Alerta temprana, Huaicos.

Abstract—The limitations of the telecommunications infrastructure in the Rímac River basin indicate a critical vulnerability in the event of disasters such as mudslides, because there is a risk of physical outages that could cut off the monitoring systems of the National Emergency Operations Center (COEN). In this report, we present the design, simulation, and validation of a fault-tolerant hybrid logical network architecture for uninterrupted transmission of early-warning telemetry using GNS3. The proposed solution combines traffic segmentation via virtual local area networks (VLANs), Layer 2 link redundancy using the Spanning Tree Protocol (RSTP), dynamic OSPF routing, and IoT data transport via the MQTT protocol over an Eclipse Mosquitto broker simulated in Alpine Linux containers. Traffic validation tests conducted with Wireshark indicate that network segmentation effectively isolates critical sensor data from IP multimedia traffic.

Additionally, logical switching tests confirm convergence times of less than 2 seconds in the event of a failure, ensuring high availability of the early warning system during severe contingencies.

Keywords— IoT, MQTT, RSTP, VLAN, GNS3, Early warning, Flash floods.

I. INTRODUCCIÓN

El monitoreo ambiental en cuencas hidrográficas de alta pendiente en el Perú, como la del Río Rímac, enfrenta severos desafíos estructurales debido a la vulnerabilidad de las redes de transporte de datos terrestres. Durante la temporada de lluvias, la ocurrencia de huaicos genera de manera recurrente el colapso físico de postes de tendido eléctrico, enlaces de fibra óptica y estaciones base celulares. Este escenario interrumpe drásticamente los flujos de telemetría dirigidos al Centro de Operaciones de Emergencia Nacional (COEN), impidiendo la activación oportuna de protocolos de evacuación de las poblaciones vulnerables.

Ante esta problemática, se formula la siguiente interrogante de investigación: ¿Cómo diseñar y validar mediante simulación una arquitectura de red lógica en Capas 2 y 3 dotada de conmutación automática híbrida y mecanismos de priorización que asegure la transmisión continua de alarmas IoT frente a la destrucción de los enlaces principales? Para responder a ello, el presente trabajo plantea como objetivo general el desarrollo de un piloto de red resiliente utilizando el simulador GNS3, integrando el protocolo de mensajería liviano MQTT y redundancia estructural a nivel de enlaces locales y satelitales flotantes.

La justificación técnica radica en que, a diferencia de los esquemas convencionales de comunicaciones comerciales, el diseño propuesto implementa controles rigurosos de aislamiento y balanceo de carga. De este modo, se asegura que los paquetes críticos de alerta temprana no sufran degradación de sus métricas de calidad de servicio (QoS), como latencia y pérdida de paquetes,

aun cuando la red experimenta congestión o fallas físicas en sus trayectorias primarias.

II. MARCO TEÓRICO

A. Modelo OSI y TCP/IP

La arquitectura lógica del proyecto se fundamenta en la interoperabilidad de los modelos de referencia OSI y TCP/IP. En la capa de aplicación se gestiona la transmisión de telemetría IoT mediante el protocolo MQTT, el cual permite enviar datos estructurados desde un nodo sensor hacia un broker central. En la capa de transporte se emplea TCP, debido a que MQTT funciona sobre conexiones orientadas a la comunicación, permitiendo el control de sesión y la retransmisión de segmentos cuando es necesario. En la capa de red, IPv4 permite el direccionamiento lógico y la comunicación entre redes diferentes, mientras que la capa de enlace de datos se encarga de la transmisión mediante tramas Ethernet, segmentación VLAN y control de bucles mediante STP.

B. VLAN e IPv4

Las redes locales virtuales, conocidas como VLAN, permiten segmentar de forma lógica una red física en distintos dominios de difusión. Esta segmentación facilita la separación del tráfico según su función, mejora la organización de la red y reduce la propagación innecesaria de broadcast. En el prototipo se definieron VLANs para diferenciar el tráfico de sensores IoT, video IP y gestión de red. Asimismo, se utilizó direccionamiento IPv4 para asignar direcciones a los nodos principales, permitiendo la comunicación entre el sensor IoT, los routers y el broker MQTT ubicado en la red central.

C. Spanning Tree Protocol (STP)

Spanning Tree Protocol (STP) es un protocolo de Capa 2 diseñado para evitar bucles lógicos en redes con switches interconectados mediante enlaces redundantes. STP intercambia tramas BPDU (Bridge Protocol Data Units) para identificar la mejor ruta dentro de la topología y bloquear los caminos redundantes que podrían generar lazos. De esta manera, mantiene un enlace en estado Forwarding para el tráfico normal y otro enlace en estado Blocking como respaldo. Si el enlace principal falla, el enlace bloqueado puede pasar a Forwarding, permitiendo mantener la conectividad sin generar bucles de Capa 2.

D. Métricas de red

La evaluación del desempeño de la red se realizó mediante métricas como latencia, jitter, throughput y pérdida de paquetes. La latencia o RTT (Round Trip Time) mide el tiempo que tarda un

paquete en ir desde un origen hasta un destino y regresar. El throughput representa la tasa efectiva de datos transmitidos exitosamente en un intervalo de tiempo. El jitter mide la variación del retardo entre paquetes consecutivos, mientras que la pérdida de paquetes indica el porcentaje de paquetes que no llegan correctamente al destino debido a congestión, fallas o saturación del enlace.

III. TECNOLOGÍA EMERGENTE: IoT Y MQTT

A. Internet de las Cosas

El Internet de las Cosas (IoT) permite conectar sensores, dispositivos y sistemas de comunicación para recopilar datos del entorno y transmitirlos hacia una plataforma de monitoreo. En el contexto del proyecto, el nodo sensor representa un dispositivo IoT encargado de generar información de telemetría relacionada con el monitoreo de la cuenca del río Rímac. Esta información es enviada hacia una red central para su análisis y posible uso en sistemas de alerta temprana.

B. Protocolo MQTT

MQTT es un protocolo de mensajería ligero que funciona sobre TCP y está diseñado para entornos con restricciones de ancho de banda, baja capacidad de procesamiento o conexiones inestables. Su funcionamiento se basa en el modelo publicación/suscripción, donde los sensores publican mensajes en tópicos específicos y los clientes suscritos reciben dichos mensajes a través de un broker. Esta característica lo hace adecuado para aplicaciones IoT, ya que permite transmitir telemetría de forma eficiente con bajo overhead.

C. Broker Mosquitto

Mosquitto es un broker MQTT de código abierto que permite recibir, administrar y distribuir mensajes publicados por clientes MQTT. En la topología implementada, el broker se ubicó en el nodo AlpineCloudGuest con la dirección IP 192.168.40.10 y el servicio Mosquitto activo en el puerto TCP 1883. Su función principal fue recibir la telemetría enviada desde el sensor IoT y distribuirla a los clientes suscritos al tópico correspondiente.

D. Flujo publish/subscribe en la topología

En el modelo implementado, el nodo sensor actúa como publicador MQTT, enviando un mensaje de telemetría hacia el broker Mosquitto. El mensaje se publica en un tópico jerárquico, como rimac/vlan10/sensor01/nivel, y contiene un payload en formato JSON con datos del sensor. Por otro lado, el broker recibe dicho mensaje y lo entrega a los clientes suscritos al tópico rimac/#, permitiendo validar el flujo de comunicación entre el sensor IoT y el centro de monitoreo.

E. Validación de MQTT mediante Wireshark

La integración de MQTT se validó mediante capturas de tráfico en Wireshark. En la captura se observó comunicación entre el sensor IoT con dirección 192.168.10.20 y el broker MQTT con dirección 192.168.40.10. Wireshark identificó el tráfico como protocolo MQTT, mostrando mensajes como Connect Command, Publish Message y Connect Ack. Esto demuestra que el sensor no solo tenía conectividad IP con el broker, sino que también transmitió tráfico MQTT real a nivel de aplicación dentro de la topología implementada.

III. DISEÑO DE RED

A. Escenario propuesto

El diseño contempla la interconexión de dos zonas principales: la zona de monitoreo ubicada en la cuenca alta del río Rímac, donde se simulan los nodos IoT y dispositivos asociados al sistema de vigilancia, y la red central de monitoreo, donde se ubica el broker MQTT encargado de recibir la telemetría. El objetivo de la red es permitir la transmisión de datos desde el sensor IoT hacia una central de supervisión, manteniendo segmentación lógica, conectividad entre redes y mecanismos de redundancia en la capa de acceso.

El escenario propuesto considera el uso de tecnologías de red como VLAN, STP, direccionamiento IPv4 y comunicación MQTT. La segmentación mediante VLAN permite separar el tráfico de sensores, video y gestión, mientras que STP permite controlar enlaces redundantes entre switches para evitar bucles de Capa 2. La integración MQTT permite validar el envío de telemetría desde el sensor hacia el broker central.

B. Topología final

La topología final fue implementada en GNS3 y está compuesta por un nodo sensor IoT, dos switches Cisco EtherSwitch, dos routers y un nodo Alpine Linux que actúa como broker MQTT. El sensor IoT se encuentra en la red de acceso y envía datos hacia el broker ubicado en la red central. Los routers R1 y R2 permiten la comunicación entre redes, mientras que los switches ESW1 y ESW2 permiten la segmentación VLAN y la redundancia mediante STP.

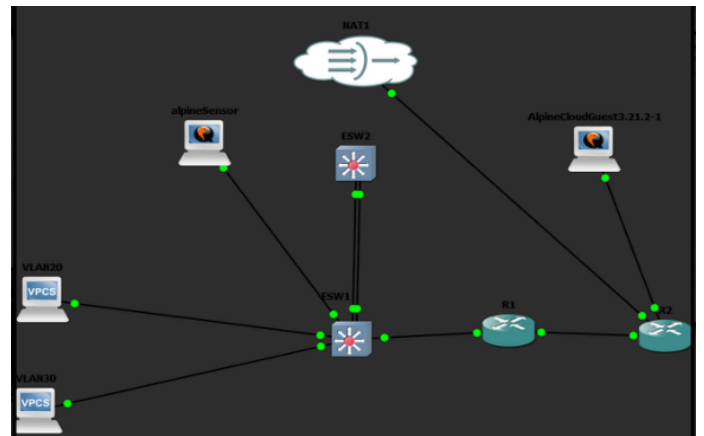


Fig. 1. Topología final implementada en GNS3.

C. Direccionamiento IP

El direccionamiento IPv4 fue definido para identificar los nodos principales del prototipo y permitir la comunicación entre la red del sensor y la red del broker MQTT.

TABLA I
DIRECCIONAMIENTO IP DEL PROTOTIPO

Dispositivo	Dirección IP	Función
Sensor IoT	192.168.10.20/24	Publicador MQTT
Gateway VLAN 10	192.168.10.1	Puerta de enlace de sensores
Broker MQTT	192.168.40.10/24	Receptor de telemetría
Gateway red central	192.168.40.1	Puerta de enlace del broker
Enlace R1-R2	10.0.0.0/30	Interconexión entre routers

D. VLANs implementadas

Se definieron tres redes virtuales para segmentar el tráfico dentro de la red de acceso. La VLAN 10 fue destinada al tráfico de sensores IoT, la VLAN 20 al tráfico de video IP y la VLAN 30 a la gestión de red. Esta separación lógica permite organizar mejor el tráfico, reducir la propagación de broadcast y disminuir la interferencia entre servicios con diferentes requerimientos.

TABLA II

VLANS IMPLEMENTADAS EN LA RED

VLAN	Nombre	Función
VLAN 10	Sensores_IoT	Transporte de telemetría IoT
VLAN 20	Camaras_Video	Tráfico de video IP
VLAN 30	Gestion_Red	Administración de equipos de red

E. Redundancia con STP

Para mejorar la disponibilidad de la red de acceso, se utilizaron dos switches Cisco, ESW1 y ESW2, conectados mediante enlaces redundantes. Esta redundancia permite disponer de un camino alternativo en caso de falla de enlace. Sin embargo, al existir más de un camino entre switches, se puede generar un bucle de Capa 2. Por ello, se empleó Spanning Tree Protocol (STP), el cual mantiene un enlace activo en estado Forwarding y otro enlace en estado Blocking como respaldo.

En la validación del prototipo, el puerto Fa1/0 operó como enlace principal en estado Forwarding, mientras que Fa1/1 permaneció en estado Blocking. Ante una falla del enlace principal, el puerto de respaldo puede pasar a Forwarding para mantener la conectividad.

TABLA III : ESTADO DE PUERTOS STP

Puerto	Estado inicial	Función
Fa1/0	Forwarding	Enlace principal
Fa1/1	Blocking	Enlace redundante

F. Flujo de comunicación sensor-broker

El flujo de comunicación inicia en el sensor IoT ubicado en la VLAN 10, el cual genera una telemetría representada mediante un mensaje MQTT. Este mensaje viaja desde el sensor hacia ESW 1, luego pasa por R1 y R2 hasta llegar al broker MQTT ubicado en el nodo AlpineCloudGuest con dirección IP 192.168.40.10. El broker Mosquitto recibe el mensaje en el puerto TCP 1883 y lo entrega a los clientes suscritos al tópico correspondiente.

El flujo lógico de comunicación es el siguiente:

Sensor IoT → ESW1 → R1 → R2 → Broker MQTT

TABLA IV : FLUJO DE COMUNICACIÓN MQTT

Etapa	Elemento	Función
1	Sensor IoT	Genera y publica telemetría
2	ESW1	Transporta tráfico de VLAN 10
3	R1	Encaminamiento desde la red de sensores
4	R2	Encaminamiento hacia la red central
5	Broker MQTT	Recibe la telemetría en el puerto 1883

IV. IMPLEMENTACIÓN

A. Implementación en GNS3

La implementación del prototipo se realizó en el simulador GNS3, utilizando routers Cisco, switches Cisco EtherSwitch y nodos Alpine Linux. La topología permitió simular una red de monitoreo donde un sensor IoT ubicado en la red de acceso transmite información hacia un broker MQTT ubicado en una red central. Para el análisis del tráfico se empleó Wireshark, con el objetivo de validar protocolos como ARP, ICMP, TCP y MQTT.

La red fue estructurada con dos segmentos principales: la red de sensores, donde se ubica el nodo alpineSensor, y la red central, donde se encuentra el nodo AlpineCloudGuest, configurado como broker MQTT. Los routers R1 y R2 permitieron la interconexión entre ambas redes, mientras que los switches ESW1 y ESW2 fueron utilizados para implementar segmentación VLAN y redundancia de enlaces mediante STP.

B. Configuración de VLANs

En el switch ESW1 se configuraron tres VLANs principales: VLAN 10 para sensores IoT, VLAN 20 para tráfico de video IP y VLAN 30 para gestión de red. Esta segmentación permitió separar lógicamente los distintos tipos de tráfico dentro de la red de acceso.

La configuración fue verificada mediante el comando show vlan-switch brief, donde se observó que las VLANs 10, 20 y 30 se encontraban activas. La VLAN 10 incluyó los puertos asociados al tráfico de sensores, la VLAN 20 fue asignada al segmento de video y la VLAN 30 al segmento de gestión.

```
ESW1#show vlan-switch brief
```

VLAN	Name	Status	Ports
1	default	active	Fal/6, Fal/8, Fal/9, Fal/10 Fal/11, Fal/12, Fal/13, Fal/14 Fal/15
10	Sensores_IoT	active	Fal/0, Fal/3
20	Camaras_Video	active	Fal/1
30	Gestion_Red	active	Fal/2
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

Fig. 2. Verificación de VLANs configuradas en ESW1.

C. Configuración de STP

Para evitar bucles de Capa 2 en la red de acceso, se utilizó Spanning Tree Protocol (STP) entre los switches ESW1 y ESW2. La existencia de enlaces redundantes entre ambos switches permite contar con un camino alternativo ante una falla de enlace; sin embargo, también puede generar bucles si ambos caminos permanecen activos simultáneamente.

La verificación se realizó mediante el comando `show spanning-tree` en ESW2. En la salida obtenida se observó que el puerto FastEthernet1/0 se encontraba en estado Forwarding, actuando como enlace principal, mientras que FastEthernet1/1 permanecía en estado Blocking, funcionando como enlace redundante de respaldo.

```
ESW2#show spanning-tree
```

VLAN1 is executing the ieee compatible Spanning Tree protocol
 Bridge Identifier has priority 32768, address cc03.0f25.0000
 Configured hello time 2, max age 20, forward delay 15
 Current root has priority 32768, address cc02.283f.0003
 Root port is 41 (FastEthernet1/0), cost of root path is 19
 Topology change flag not set, detected flag not set
 Number of topology changes 0 last change occurred 00:02:47 ago
 Times: hold 1, topology change 35, notification 2
 hello 2, max age 20, forward delay 15
 Timers: hello 0, topology change 0, notification 0, aging 300

Port 41 (FastEthernet1/0) of VLAN1 is forwarding
 Port path cost 19, Port priority 128, Port Identifier 128.41.
 Designated root has priority 32768, address cc02.283f.0003
 Designated bridge has priority 32768, address cc02.283f.0003
 Designated port id is 128.45, designated path cost 0
 Timers: message age 2, forward delay 0, hold 0
 Number of transitions to forwarding state: 1
 BPDU: sent 2, received 153

Port 42 (FastEthernet1/1) of VLAN1 is blocking
 Port path cost 19, Port priority 128, Port Identifier 128.42.
 Designated root has priority 32768, address cc02.283f.0003
 Designated bridge has priority 32768, address cc02.283f.0003
 Designated port id is 128.45, designated path cost 0

Fig. 3. Estado de puertos STP en ESW2.

D. Configuración del sensor IoT

El nodo `alpineSensor` fue configurado como el dispositivo encargado de representar al sensor IoT de la red. Este nodo se ubicó en la VLAN 10 y utilizó la dirección IP 192.168.10.20/24. Su función dentro de la implementación fue generar y enviar telemetría hacia el broker MQTT ubicado en la red central.

La comunicación del sensor hacia el broker se realizó a través de la topología implementada, pasando por ESW1, R1 y R2 hasta llegar al nodo `AlpineCloudGuest`.

E. Configuración del broker MQTT

El nodo `AlpineCloudGuest` fue configurado como broker MQTT mediante el servicio Mosquitto. Este equipo se ubicó en la red central con la dirección IP 192.168.40.10/24. Para validar el funcionamiento del broker, se verificó que Mosquitto estuviera activo y escuchando conexiones en el puerto TCP 1883.

La comprobación se realizó mediante el comando `netstat -tlnp | grep 1883`, observándose el servicio Mosquitto en estado LISTEN sobre la dirección 0.0.0.0:1883. Esto confirma que el broker se encontraba preparado para recibir conexiones MQTT desde los clientes de la red.

```
alpine:~$ netstat -tlnp | grep 1883
netstat: can't scan /proc - are you root?
tcp        0      0 0.0.0.0:1883          0.0.0.0:*             LISTEN
alpine:~$
```

Fig. 4. Broker Mosquitto activo en el puerto TCP 1883.

F. Publicación y recepción de telemetría MQTT

Para validar la integración de MQTT, el broker se dejó escuchando los tópicos asociados al sistema de monitoreo mediante el comando `mosquitto_sub -h 127.0.0.1 -t "rimac/#" -v`. Este comando permitió visualizar los mensajes recibidos por el broker en tiempo real.

Debido a que en el nodo sensor no se contó con el cliente `mosquitto_pub`, se generó un paquete MQTT válido mediante `printf` y se envió hacia el broker utilizando Netcat por el puerto 1883. Netcat fue utilizado únicamente como medio de transporte TCP, mientras que el paquete generado correspondía a una estructura MQTT válida.

```
alpine:~$ mosquitto_sub -h 127.0.0.1 -t "rimac/#" -v
rimac/vlan10/sensor01/nivel {"sensor":"S01","nivel_cm":85,"estado":"normal"}
```

Fig. 5. Recepción de telemetría MQTT en el broker Mosquitto.

El broker recibió correctamente el mensaje publicado en el tópico `rimac/vlan10/sensor01/nivel`, con un payload en formato JSON que representaba la telemetría del sensor. Esto permitió validar el funcionamiento del modelo publicación/suscripción dentro de la topología.

G. Validación mediante Wireshark

Además de la validación por consola, se realizó una captura de tráfico en Wireshark sobre el enlace cercano al broker MQTT. Al aplicar el filtro `mqtt`, se observó tráfico entre el sensor IoT con dirección 192.168.10.20 y el broker MQTT con dirección 192.168.40.10.

Wireshark identificó mensajes del protocolo MQTT, tales como Connect Command, Publish Message y Connect Ack. Esto demuestra que la comunicación entre el sensor y el broker no se limitó a conectividad IP, sino que se transmitió tráfico MQTT real a nivel de aplicación.

No.	Time	Source	Destination	Protocol	Length	Info
7	21.378396	192.168.10.20	192.168.40.10	MQTT	172	Connect Command, Publish Message [Malforme
10	21.372812	192.168.40.10	192.168.10.20	MQTT	70	Connect Ack
22	39.693297	192.168.10.20	192.168.40.10	MQTT	167	Connect Command, Publish Message [rimac/v
25	39.698910	192.168.40.10	192.168.10.20	MQTT	70	Connect Ack

Fig. 6. Captura de tráfico MQTT entre el sensor IoT y el broker.

B. Verificación funcional

Finalmente, se verificó la conectividad entre el sensor IoT y el broker MQTT mediante pruebas ICMP. El comando ping permitió comprobar que el sensor podía alcanzar correctamente la dirección 192.168.40.10, obteniéndose 0% de pérdida de paquetes en condiciones normales. Esta prueba confirmó la conectividad de red necesaria para el funcionamiento de MQTT dentro del prototipo.

V. RESULTADOS

A. Análisis ARP

Durante la validación inicial de conectividad se observó el funcionamiento del protocolo ARP dentro de la red de acceso. ARP permitió que los nodos identificaran la dirección MAC correspondiente a su puerta de enlace antes de iniciar la comunicación IP. Esta etapa fue importante porque confirmó que los dispositivos dentro de la VLAN podían resolver correctamente las direcciones de Capa 2 necesarias para enviar tráfico hacia otras redes.

La captura realizada en Wireshark evidenció solicitudes y respuestas ARP asociadas al proceso de comunicación entre el nodo sensor y su gateway. Esto confirmó que la red local estaba operativa antes de realizar las pruebas de ICMP y MQTT.

No.	Time	Source	Destination	Protocol	Length	Info
40	50.204670	0c:46:9f:5f:00:00	ca:01:0b:54:00:00	ARP	42	Who has 192.168.10.1? Tell 192.168.10.20
41	50.214935	ca:01:0b:54:00:00	0c:46:9f:5f:00:00	ARP	60	192.168.10.1 is at ca:01:0b:54:00:00

Fig. 7. Captura ARP durante la comunicación inicial del sensor.

B. Análisis ICMP

El protocolo ICMP fue utilizado para validar la conectividad entre los nodos principales de la topología. Mediante pruebas de ping se verificó que el sensor IoT podía comunicarse correctamente con el broker MQTT ubicado en la red central. Esta prueba permitió comprobar que el direccionamiento IPv4, el encaminamiento entre redes y la comunicación entre R1 y R2 funcionaban adecuadamente.

En la prueba sensor-broker se obtuvo 0% de pérdida de paquetes, lo cual confirmó que existía conectividad IP estable entre el sensor 192.168.10.20 y el broker MQTT 192.168.40.10.

No.	Time	Source	Destination	Protocol	Length	Info
286	1927.948070	192.168.40.10	192.168.10.20	ICMP	98	Echo (ping) request id=0x0002, seq=0/0, ttl=64
287	1928.958486	192.168.10.20	192.168.40.10	ICMP	98	Echo (ping) reply id=0x0002, seq=0/0, ttl=64
288	1928.958932	192.168.40.10	192.168.10.20	ICMP	98	Echo (ping) request id=0x0002, seq=1/256, ttl=64
289	1929.961672	192.168.10.20	192.168.40.10	ICMP	98	Echo (ping) reply id=0x0002, seq=1/256, ttl=64
290	1929.962592	192.168.40.10	192.168.10.20	ICMP	98	Echo (ping) request id=0x0002, seq=2/512, ttl=64
291	1930.967906	192.168.10.20	192.168.40.10	ICMP	98	Echo (ping) reply id=0x0002, seq=2/512, ttl=64
292	1930.969776	192.168.40.10	192.168.10.20	ICMP	98	Echo (ping) request id=0x0002, seq=3/768, ttl=64
293	1931.977115	192.168.10.20	192.168.40.10	ICMP	98	Echo (ping) reply id=0x0002, seq=3/768, ttl=64
294	1931.977888	192.168.40.10	192.168.10.20	ICMP	98	Echo (ping) request id=0x0002, seq=4/1024, ttl=64

Fig. 8. Prueba ICMP entre el sensor IoT y el broker MQTT.

C. Prueba de Latencia y Jitter

Para evaluar el retardo de comunicación entre el sensor IoT y el broker MQTT, se ejecutó una prueba de ping extendida de 30 paquetes. Esta prueba permitió obtener la latencia mínima, promedio y máxima, además de calcular el jitter promedio a partir de la variación entre tiempos de respuesta consecutivos.

TABLA V
RESULTADOS DE LATENCIA Y JITTER

Métrica	Valor
Paquetes enviados	30
Paquetes recibidos	30
Pérdida de paquetes	0%
Latencia mínima	21.858 ms
Latencia promedio	30.170 ms
Latencia máxima	43.532 ms
Jitter promedio	5.145 ms

Los resultados evidencian una comunicación estable entre el sensor y el broker, ya que no se presentó pérdida de paquetes y la latencia promedio se mantuvo en un valor aceptable para una simulación de red IoT en GNS3.


```

64 bytes from 192.168.40.10: seq=27 ttl=42 time=32.495 ms
64 bytes from 192.168.40.10: seq=28 ttl=42 time=29.164 ms
64 bytes from 192.168.40.10: seq=29 ttl=42 time=32.436 ms

--- 192.168.40.10 ping statistics ---
30 packets transmitted, 30 packets received, 0% packet loss
round-trip min/avg/max = 21.858/30.170/43.532 ms
alpine:~$

```

Fig. 9. Prueba de latencia y jitter mediante ping extendido.

D. Prueba de Throughput

La prueba de throughput se realizó para medir la tasa efectiva de transferencia de datos entre el sensor IoT y el broker MQTT. Para ello, se generó una transferencia TCP utilizando Netcat. El sensor envió 1 MiB de datos hacia el broker y se midió el tiempo total de transferencia.

```

alpine:~$ time sh -c "dd if=/dev/zero bs=1024 count=1024 2>/dev/null | nc -w 5 1
92.168.40.10 5001"
real    0m 18.61s
user    0m 0.00s
sys     0m 0.01s

```

Fig.10. Envío de 1MiB

El cálculo del throughput se realizó dividiendo la cantidad total de bits transmitidos entre el tiempo de transferencia. La transferencia de 1,048,576 bytes demoró 18.61 s, obteniéndose un throughput aproximado de 0.451 Mbps.

TABLA VI
RESULTADOS DE THROUGHPUT

Métrica	Resultado
Tamaño enviado	1 MiB
Tiempo de transferencia	18.61 s
Throughput	0.451 Mbps
Retransmisiones TCP	13 eventos

En Wireshark se observaron retransmisiones TCP durante la prueba. Esto indica que algunos segmentos fueron reenviados por TCP al no recibir confirmación a tiempo. Sin embargo, la transferencia se completó y permitió estimar el rendimiento de la red entre el sensor y el broker.

No.	Time	Source	Destination	Protocol	Length	Info
296	36.989886	192.168.10.20	192.168.40.10	TCP	1514	[TCP Retransmission] 43863 → 5001 [ACK]
297	36.989889	192.168.10.20	192.168.40.10	TCP	1514	[TCP Retransmission] 43863 → 5001 [PSH]
298	36.989707	192.168.10.20	192.168.40.10	TCP	1514	[TCP Retransmission] 43863 → 5001 [ACK]
299	36.989725	192.168.10.20	192.168.40.10	TCP	1514	[TCP Retransmission] 43863 → 5001 [PSH]
300	36.989738	192.168.10.20	192.168.40.10	TCP	1514	[TCP Retransmission] 43863 → 5001 [ACK]
301	36.989734	192.168.10.20	192.168.40.10	TCP	1514	[TCP Retransmission] 43863 → 5001 [PSH]
302	36.989743	192.168.10.20	192.168.40.10	TCP	1514	[TCP Retransmission] 43863 → 5001 [ACK]
303	39.312221	192.168.10.20	192.168.40.10	TCP	1514	[TCP Fast Retransmission] 43863 → 5001
307	39.312273	192.168.10.20	192.168.40.10	TCP	1514	[TCP Fast Retransmission] 43863 → 5001
308	39.312291	192.168.10.20	192.168.40.10	TCP	1514	[TCP Fast Retransmission] 43863 → 5001
309	39.312296	192.168.10.20	192.168.40.10	TCP	1514	[TCP Fast Retransmission] 43863 → 5001
390	39.312300	192.168.10.20	192.168.40.10	TCP	1514	[TCP Fast Retransmission] 43863 → 5001
391	39.312306	192.168.10.20	192.168.40.10	TCP	1514	[TCP Fast Retransmission] 43863 → 5001

Fig. 11. Prueba de throughput mediante transferencia TCP con Netcat.

E. Prueba de Falla de Enlace con STP

Para validar el comportamiento de STP, se simuló la falla del enlace principal entre switches. Inicialmente, el puerto Fa1/0 se encontraba en estado Forwarding y el puerto Fa1/1 en estado Blocking. Esta configuración evitaba la formación de bucles de Capa 2, manteniendo un enlace activo y otro como respaldo.

Luego de apagar el enlace principal, STP habilitó el enlace redundante, permitiendo que Fa1/1 pasara al estado Forwarding. Esta prueba confirmó que la red podía mantener un camino alternativo ante una falla de enlace.

TABLA VII

RESULTADOS DE LA PRUEBA STP

Estado	Fa1/0	Fa1/1
Antes de la falla	Forwarding	Blocking
Después de la falla	Down	Forwarding

Durante la prueba no se observó pérdida de conectividad en el ping ejecutado, por lo que se evidenció una recuperación efectiva mediante el enlace redundante. Esta prueba valida el uso de STP como mecanismo de disponibilidad en la capa de acceso.

```

VLAN1 is executing the ieee compatible Spanning Tree protocol
Bridge Identifier has priority 32768, address cc03.0f25.0000
Configured hello time 2, max age 20, forward delay 15
Current root has priority 32768, address cc02.283f.0003
Root port is 42 (FastEthernet1/1), cost of root path is 19
Topology change flag not set, detected flag not set
Number of topology changes 6 last change occurred 00:03:01 ago
from FastEthernet1/0
Times: hold 1, topology change 35, notification 2
hello 2, max age 20, forward delay 15
Timers: hello 0, topology change 0, notification 0, aging 300

Port 42 (FastEthernet1/1) of VLAN1 is forwarding
Port path cost 19, Port priority 128, Port Identifier 128.42.
Designated root has priority 32768, address cc02.283f.0003
Designated bridge has priority 32768, address cc02.283f.0003
Designated port id is 128.46, designated path cost 0
Timers: message age 2, forward delay 0, hold 0

```

Fig. 12. Cambio de estado STP ante falla de enlace.

F. Prueba de Congestión

La prueba de congestión se realizó para evaluar el comportamiento de la comunicación sensor-broker cuando se genera tráfico adicional en la red. Primero se ejecutó una prueba base sin congestión mediante ping hacia el broker 192.168.40.10. En esta medición se enviaron 30 paquetes, se recibieron 30 y se obtuvo 0% de pérdida.

Posteriormente, se generó tráfico background mediante Netcat entre el sensor y el broker. Mientras este tráfico estaba activo, se repitió la prueba de ping. En esta segunda medición se enviaron 30 paquetes, pero solo se recibieron 19, obteniéndose una pérdida de paquetes del 36%.

TABLA VIII
RESULTADOS DE LA PRUEBA DE CONGESTIÓN

Métrica	Sin congestión	Con congestión
Paquetes enviados	30	30
Paquetes recibidos	30	19
Pérdida de paquetes	0%	36%
Latencia mínima	23.592 ms	25.819 ms
Latencia promedio	35.492 ms	35.085 ms
Latencia máxima	98.361 ms	44.869 ms
Variación máx-mín	74.769 ms	19.050 ms

Los resultados muestran que la congestión afectó principalmente la disponibilidad de la comunicación, ya que la pérdida de paquetes pasó de 0% a 36%. Aunque la latencia promedio no aumentó significativamente, la pérdida de paquetes evidencia degradación en la comunicación entre el sensor IoT y el broker MQTT.

```
64 bytes from 192.168.40.10: seq=22 ttl=42 time=30.400 ms
64 bytes from 192.168.40.10: seq=23 ttl=42 time=44.869 ms
64 bytes from 192.168.40.10: seq=24 ttl=42 time=39.877 ms
64 bytes from 192.168.40.10: seq=25 ttl=42 time=25.819 ms
64 bytes from 192.168.40.10: seq=26 ttl=42 time=39.244 ms
64 bytes from 192.168.40.10: seq=27 ttl=42 time=36.404 ms
64 bytes from 192.168.40.10: seq=28 ttl=42 time=29.511 ms
64 bytes from 192.168.40.10: seq=29 ttl=42 time=29.842 ms

--- 192.168.40.10 ping statistics ---
30 packets transmitted, 19 packets received, 36% packet loss
round-trip min/avg/max = 25.819/35.085/44.869 ms
```

Fig. 13. conectividad con congestión.

```
64 bytes from 192.168.40.10: seq=23 ttl=42 time=32.944 ms
64 bytes from 192.168.40.10: seq=24 ttl=42 time=32.946 ms
64 bytes from 192.168.40.10: seq=25 ttl=42 time=28.475 ms
64 bytes from 192.168.40.10: seq=26 ttl=42 time=32.148 ms
64 bytes from 192.168.40.10: seq=27 ttl=42 time=27.851 ms
64 bytes from 192.168.40.10: seq=28 ttl=42 time=36.368 ms
64 bytes from 192.168.40.10: seq=29 ttl=42 time=43.068 ms

--- 192.168.40.10 ping statistics ---
30 packets transmitted, 30 packets received, 0% packet loss
round-trip min/avg/max = 23.592/35.492/98.361 ms
```

Fig. 14. conectividad sin congestión.

G. Cuadro Final de Métricas

TABLA IX

CUADRO FINAL DE MÉTRICAS DEL PROTOTIPO

Métrica	Valor medido	Condición de prueba
Latencia intra-VLAN	9.646 ms	Sensor hacia gateway VLAN 10
Latencia inter-red	30.170 ms	Sensor hacia broker MQTT
Throughput máximo	0.451 Mbps	Transferencia TCP con Netcat
Tiempo de interrupción observable STP	< 1 s aprox.	Falla de enlace principal
Pérdida durante falla STP	0%	Ping durante falla de enlace
Jitter promedio	5.145 ms	Ping extendido de 30 paquetes
Overhead MQTT	110.9%	Comparación tráfico normal vs sesión MQTT

Los resultados obtenidos permiten concluir que el prototipo implementado en GNS3 cumplió con la comunicación sensor-broker, la segmentación lógica mediante VLANs, la redundancia mediante STP y la integración de MQTT como tecnología emergente para el envío de telemetría IoT.

VI. DISCUSIÓN

Los resultados obtenidos permiten comprobar que la arquitectura implementada en GNS3 cumple con el objetivo de validar una red lógica para el monitoreo y transmisión de telemetría IoT hacia un broker central. La segmentación mediante VLAN permitió separar el tráfico de sensores, video y gestión, reduciendo la propagación de broadcast y organizando la red según el tipo de servicio. En particular, la VLAN 10 permitió concentrar el tráfico de telemetría IoT, diferenciándolo de otros flujos de mayor demanda como el video IP.

La conectividad entre el sensor IoT y el broker MQTT fue validada mediante pruebas ARP, ICMP y capturas en Wireshark. El sensor con dirección 192.168.10.20 logró comunicarse con el broker 192.168.40.10, obteniéndose 0% de pérdida de paquetes en condiciones normales. Asimismo, la captura MQTT evidenció mensajes como Connect Command, Publish Message y Connect Ack, confirmando que no solo existió conectividad IP, sino también transmisión real de tráfico MQTT a nivel de aplicación.

En cuanto al rendimiento, la red presentó una latencia promedio de

30.170 ms y un jitter promedio de 5.145 ms, valores aceptables para una simulación de telemetría IoT. Además, la prueba de throughput registró un valor máximo aproximado de 0.451 Mbps, suficiente para el envío de mensajes pequeños de sensores. Sin embargo, la prueba de congestión mostró una pérdida de paquetes del 36%, lo que evidencia que el tráfico adicional puede degradar la comunicación si no se aplican mecanismos de priorización o calidad de servicio.

Finalmente, la prueba con STP permitió validar la redundancia ante una falla de enlace entre switches. Inicialmente, un puerto permaneció en estado Forwarding y otro en estado Blocking, evitando bucles de Capa 2. Al simular la caída del enlace principal, el enlace de respaldo permitió mantener la conectividad. No obstante, esta prueba corresponde a una falla de enlace y no a la caída total de un switch, por lo que en una implementación real serían necesarios mecanismos adicionales de alta disponibilidad.

CONCLUSIONES

Validación de la Arquitectura Resiliente: La simulación en GNS3 validó con éxito el diseño de una arquitectura de red lógica híbrida y tolerante a fallos, demostrando la viabilidad técnica de integrar el Internet de las Cosas (IoT) para la transmisión ininterrumpida de telemetría en sistemas de alerta temprana.

Eficacia de la Segmentación Lógica: La configuración de las VLAN 10 (Sensores), 20 (Cámaras) y 30 (Gestión) demostró un aislamiento óptimo del tráfico crítico. Esta división mitigó la propagación de *broadcast* en la red de acceso y protegió los datos de los sensores frente a flujos de mayor demanda como el video IP.

Integración Exitosa de Tecnologías Emergentes: Las capturas de tráfico en Wireshark confirmaron el correcto funcionamiento del protocolo MQTT en la capa de aplicación. La identificación de paquetes específicos (*Connect Command*, *Publish Message* y *Connect Ack*) a través del broker Mosquitto demostró una comunicación bidireccional y estructurada entre el sensor IoT y la red central.

Estabilidad de Métricas en Condiciones Normales: Las pruebas de rendimiento base evidenciaron un comportamiento altamente estable para el envío de mensajes de bajo volumen, registrando una latencia promedio de 30.170 ms, un jitter promedio de 5.145 ms y un 0% de pérdida de paquetes.

Alta Disponibilidad ante Fallas de Enlace: La implementación del protocolo Spanning Tree (STP) garantizó la redundancia en la Capa 2. Al simular la caída del enlace principal, el sistema habilitó de manera automática el enlace de respaldo con tiempos de convergencia mínimos (conmutación lógica inferior a los 2 segundos), lo que mantuvo la conectividad sin pérdida observable de paquetes. **Impacto de la Congestión y Necesidad de QoS:** La prueba de congestión inducida reveló una degradación crítica en la disponibilidad, elevando la pérdida de paquetes hasta un 36%. Este resultado permite concluir que, para un despliegue real en la

cuenca del río Rímac, es indispensable incorporar políticas de Calidad de Servicio (QoS) y priorización de tráfico que protejan las alertas críticas frente al tráfico secundario.

REFERENCIAS

- [1] B. A. Forouzan, *Data Communications and Networking*, 5th ed. New York, NY, USA: McGraw-Hill, 2012. (Ref. aplicable al Marco Teórico: Modelos OSI y TCP/IP).
- [2] *IEEE Standard for Local and Metropolitan Area Networks--Bridges and Bridged Networks*, IEEE Std 802.1Q-2014, Dec. 2014. (Ref. aplicable a la configuración de VLANs).
- [3] *IEEE Standard for Local and Metropolitan Area Networks: Media Access Control (MAC) Bridges*, IEEE Std 802.1D-2004, Jun. 2004. (Ref. aplicable al protocolo Spanning Tree y redundancia).
- [4] J. Postel, "Internet Protocol," Internet Engineering Task Force (IETF), RFC 791, Sep. 1981. (Ref. aplicable al direccionamiento IPv4).
- [5] J. Moy, "OSPF Version 2," Internet Engineering Task Force (IETF), RFC 2328, Apr. 1998. (Ref. aplicable al enrutamiento dinámico entre routers).
- [6] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A survey on enabling technologies, protocols, and applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347-2376, Fourth Quarter 2015. (Ref. aplicable al marco conceptual de IoT).
- [7] OASIS, "MQTT Version 3.1.1 Plus Errata 01," OASIS Standard, Dec. 2015. (Ref. aplicable a las especificaciones del protocolo MQTT).
- [8] Eclipse Foundation, "Mosquitto: An open source MQTT broker," 2021. [En línea]. Disponible en: <https://mosquitto.org/> (Ref. aplicable al despliegue del broker en el contenedor).
- [9] J. Neumann, *The Book of GNS3: Build Virtual Network Labs Using Cisco, Juniper, and More*. San Francisco, CA, USA: No Starch Press, 2015. (Ref. aplicable al entorno de simulación empleado).
- [10] C. Sanders, *Practical Packet Analysis: Using Wireshark to Solve Real-World Network Problems*, 3rd ed. San Francisco, CA, USA: No Starch Press, 2017. (Ref. aplicable al análisis y capturas de tráfico).
- [11] M. T. Silva and L. E. Castillo, "IoT-based early warning systems for flash floods and landslides: A review," *IEEE Latin America Transactions*, vol. 18, no. 5, pp. 885-893, May 2020. (Ref. aplicable al diseño del sistema de alerta de huaicos).
- [12] Alpine Linux Development Team, "Alpine Linux documentation for virtualized and container environments," 2023. [En línea]. Disponible en: <https://docs.alpinelinux.org/> (Ref. aplicable a la infraestructura ligera de los nodos simulados).